

NETWORK SECURITY

Project Overview

In this foundational module of the **Network Security** project, you will abandon ad-hoc scripting and adopt the workflow of a Senior ML Engineer. You will construct a robust, production-ready development environment designed for scalability and collaboration.

You will start by architecting a standard **Data Science Project Structure** using **VS Code**, creating a clean workspace that separates code, configuration, and documentation. You will master **Environment Management** using **Anaconda**, creating a strictly isolated Python 3.10 environment to prevent dependency conflicts—a notorious issue in real-world deployment.

Finally, you will lay the groundwork for **MLOps automation**. You will configure **Git** with a `.gitignore` file to ensure repository hygiene and set up the skeletal structure for **CI/CD pipelines** using **GitHub Actions**, preparing your project for automated cloud deployment from day one.

What you will Learn?

- **Project Structuring:** How to organize a Data Science project like a professional engineer, not just a notebook user.
- **Virtualization:** The critical importance of **Conda environments** for dependency isolation and reproducibility.

- **Command Line Proficiency:** Navigating file systems and executing complex commands via **Git Bash** or **CMD**.
- **Source Control Hygiene:** Understanding exactly what to commit vs. what to ignore to keep repositories clean.
- **DevOps Readiness:** Preparing a project structure that is ready for **Continuous Integration/Continuous Deployment (CI/CD)**.

Why this Project?

- **Engineering Over Modeling:** Most data science courses focus 90% on the model and 10% on the setup. This project flips that script. You learn the **Professional Engineering Standards**—directory structures, environment isolation, and version control—that distinguish a Senior Data Scientist from a hobbyist.
- **Solving "It Works on My Machine":** By mastering **Conda Environment Management** and rigorous requirements.txt handling, you solve the most common headache in deployment: dependency hell. You aren't just building a model; you are building a *reproducible system*.
- **DevOps-Ready Architecture:** You aren't treating **CI/CD** as an afterthought. By setting up GitHub Actions workflows (.github/workflows) from day one, you demonstrate that you understand the full **MLOps Lifecycle**, making this a production-grade portfolio piece.

Module
Environment & Infrastructure
Project structure Set up With Environment
Github Repository set up with Vscode
Setup .py setup
Loggign And Exception module
Data Engineering (ETL)
Introduction To ETL Pipeline
Setting Up MongoDB Atlas
ETL Pipeline Set up
Automated Data Ingestion
Data Ingestion Architecture
Data Ingestion Configuration
Implementing Data Ingestion
Validation & Transformation
Data Validation Part 1
Data Validation Part2
Data Trnasformation Architecture
Data Transformation Implementation
Training & MLFlow Tracking
Model Trainer Part 1

Model Trainer And Evaluation
Model Experiment Tracker With MLflow
MLFLOW Tacking With Dagshub
Model Deployment Strategy
Model Pusher
Model Training Pipeline Implementation
Batch Prediction
Model And Artifact Pusher To S3
CI/CD & Cloud Orchestration
Docker Image And Github Action
Docker Image Build And Push ECR
Deployment To EC2 Instance

Tech Stack & Tools

1. **IDE:** VS Code
2. **Environment Manager:** Anaconda (Conda)
3. **Language:** Python 3.10
4. **Version Control:** Git & GitHub
5. **Automation:** GitHub Actions (Configuration)
6. **Terminal:** Git Bash / Command Prompt

